

Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on $N = 120$ synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided $p = 1.0$. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39b15ba).

I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across $N = 120$ intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar $p = 1.0$); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. RELATED WORK

Verifier-grounded NetOps and intent→configuration. Offline semantics analyzers that compute network final and transient state from device configurations are established engineering controls in pre-deployment validation. Batfish and related tools demonstrate how high-fidelity configuration analysis enables operators to detect semantic errors before changes reach production; Batfish’s engineering lessons emphasize scalability, fidelity, and the operational value of deterministic verification in NetOps workflows [10.1145/3603269.3604866]. These verifiers produce machine-checkable oracles (reachability matrices, ACL equivalence, route-preservation checks and transient-state invariants) that directly map to concrete operational failure modes (lost reachability, unintended route leaks, unsafe transient downtimes).

LLM→NetOps evaluations and generator–validator patterns. Recent work on intent-driven configuration generation and related pipelines has converged on generate–validate–repair architectural patterns: an LLM proposes candidate configurations, a deterministic verifier checks semantics, and a repair loop attempts to fix validator failures. Surveys and position papers on LLM-based network management identify this pattern and emphasize that deterministic validators are necessary to move beyond syntactic or fluency-based evaluation metrics [10.1002/nem.70029]. Empirical studies of LLM repair loops in adjacent domains show diminishing returns from unbounded iteration and highlight that evaluation out-

comes depend strongly on orchestration and feedback design rather than model identity alone; these findings motivate careful treatment of repair budgets and sampling semantics when designing experiments that measure operational reliability.

Evaluation pitfalls: semantic scoring vs. generative variability. Prior evaluations have sometimes conflated different scientific questions: (i) how often a particular single candidate produced by a pipeline passes a verifier (a candidate-level success), (ii) how reliably independently sampled candidates satisfy verifier constraints (a generative-reliability probability), and (iii) whether a repair loop can raise a failed candidate to pass. Studies that report per-candidate pass rates without specifying generation semantics or seed treatment risk ambiguity. Our preregistered design explicitly fixed generation semantics (shared_initial_candidate) and thus answered a narrowly specified estimand: whether a deterministic verifier treats identical artifacts differently across scoring conditions—an important but distinct question from independent LLM reliability. The distinction maps directly onto operational decisions: operators choosing to accept a single canonical LLM candidate with verification differ from operators who rely on repeated sampling or repair loops to achieve probabilistic guarantees.

Validator–task coupling and construct validity. When tasks are programmatically templated and validators enforce the very workflow_policies encoded in those templates, canonical candidates that mirror the template structure will tend to pass deterministically. This validator–task coupling can inflate pass rates relative to open distributions of natural intents. The literature on intent-driven generators recognizes this trade-off: narrow, template-aligned tasks allow reproducible, deterministic evaluation useful for principled method comparisons, but they limit external validity to more heterogeneous operational intents (surveyed in [10.1002/nem.70029]).

Preregistration, artifact release, and reproducibility. A complementary body of work emphasizes transparent artifact publication (model traces, seeds, verifier logs) and preregistration to make explicit what a given test measures and to permit deterministic reruns under alternative semantics. Our contribution adopts this practice: we preregistered the estimand, generation semantics, and analysis plan, and we release the full per-episode scoring traces and SHA-256 manifest so that others can reexecute the experiment under different sampling semantics (for example, independent_per_condition sampling or multi-seed confirmatory designs) while preserving deterministic reproducibility of each variant. In net effect, verifier grounding plus exhaustive artifact publication clarifies both the scientific question answered and the minimal modifications required to answer adjacent operational questions.

III. METHODOLOGY

Preregistration and frozen design. The full preregistration (CAND-2026-001-NetConfig-Semantic-v1) was archived prior to any model calls (SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fde71a37e). The preregistered confirmatory design specified a paired comparison across $N = 120$ tasks (40 tasks per topology

class: edge, datacenter, multi-AS), the primary estimand paired_success_rate_difference_guarded_minus_baseline, confirmatory generation_semantics = "shared_initial_candidate", one canonical seed_0 per pair for the confirmatory analysis, the deterministic validator deterministic_netops_validator_v5 (validator_version 5.0) and the scoring rule full_intent_constraint_validation. The preregistered analysis executor was paired_binary_analysis_v1 with bootstrap inference (10,000 resamples, seed = 1729). The preregistration and analysis plan (including failure treatment, retry policy, and contamination heuristics) are published in the artifact bundle.

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: the initial_state, expected_final_state, an explicit required_sequence (the minimal safe command sequence), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation (changed_interface_count, transient_rule_count). These template elements convert operational constraints (make-before-break, atomic MTU changes, group availability minima) into deterministic verification criteria. Representative task manifests and their exact file locations are archived (execution/task_manifest.jsonl; see artifact_examples within the bundle).

Model, orchestration, and exact generation semantics. The declared LLM in the preregistration and execution manifest was gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the pinned model configuration file is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82). Per the preregistered confirmatory semantics we produced one canonical candidate per task (shared_initial_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; these canonical shared candidates are archived under execution/responses/*-shared-initial.txt (example SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...). The execution manifest records that model_calls_used = 120 while planned_episode_count = 240 (the shared_initial design reduces distinct generation calls per condition and is explicitly documented in the manifest).

Deterministic validation and scoring. Each candidate was evaluated by deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline performs: normalization and parsing of candidate commands; enforcement of per-task workflow_policies and transient constraints (e.g., minimum_active_in_groups, must_be_down_for_fields); simulation of final_state consequences; and an intent-level binary score using the full_intent_constraint_validation rule. Redeployment. JSON scoring traces (execution/scoring/*_json) include parsed_command_count, required_command_count, normalized_configuration, validation_before/after final_state

dictionaries, and violation lists. Representative scoring files are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. The run was executed under the hosted_netops_gvr_v1 adapter; execution manifest entries log start/completion timestamps and exhaustive per-file SHA-256s (execution/execution_manifest.json). Planned_episode_count = 240; completed_episode_count = 240; model_calls_used = 120 (shared_initial generation per pair counted once); failed_episode_count = 0. A preregistered retry policy allowed up to two automatic retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run per preregistration and recorded in analysis/contamination_summary.csv; no confirmatory items were flagged under the chosen thresholds (however the preregistration and Disclosure explicitly note that absence of a flag is not proof of zero pretraining exposure).

Primary inferential procedure. For each pair i ($i = 1..120$) the baseline_i and guarded_i binary verifier outcomes were recorded. The paired estimator is $\text{mean}_{\{i\}}(\text{guarded}_i - \text{baseline}_i)$. Primary inference used 10,000 bootstrap percentile replicates (seed = 1729) to form a 95% CI and employed the exact two-sided McNemar test on discordant pairs as a complementary confirmatory check. All analysis code, bootstrap parameters, and the analysis log are archived under analysis/ (see analysis/analysis_log.jsonl SHA-256 ff20bc07...).

Key artifact index (representative, for rapid audit). The following canonical artifact paths and SHA-256 fingerprints are included in the submission bundle to enable byte-for-byte reruns and immediate inspection: - preregistration: provenance/preregistration_CAND-2026-001.json (SHA-256 7db1663cf3982c8e...) - initial master prompt: provenance/master_prompt.txt (SHA-256 1872df1e1805d2d9...) - model configuration: execution/model_configuration.json (SHA-256 a40e96e212ab87da...) - execution manifest (full artifact index): execution/execution_manifest.json - shared canonical candidate (example): execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...) - scoring trace (example): execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) - analysis artifacts: analysis/condition_summary.csv (SHA-256 9c77c96f37c4b6ff...), analysis/paired_contingency_table.csv (SHA-256 9f25790c7eeafb3...).

Deviations and transparency. The methodology above follows the frozen preregistration; all deviations, retries, exclusions, and per-file SHA-256 fingerprints are recorded in the execution manifest and analysis logs. The artifact bundle provides the exact provenance needed for external auditors to re-execute the analysis under alternative generation semantics (e.g., independent_per_condition or multi-seed confirmatory designs) without introducing unspecified choices.

IV. RESULTS

Confirmatory counts and inferential summary (artifact grounding). All confirmatory scoring episodes completed and are traceable in the execution manifest (execution/execution_manifest.json). The archived confirmatory counts are: baseline_success_count = 120/120; guarded_success_count = 120/120; complete_pair_count = 120. The paired contingency table is degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$ (analysis/paired_contingency_table.csv SHA-256 9f25790c7eeafb3...). The primary estimator (mean of guarded_i - baseline_i) = 0.00. The preregistered bootstrap (10,000 resamples, seed 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided $p = 1.0$. Analysis artifacts (condition_summary.csv SHA-256 9c77c96f37c4b6ff..., paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7...) are published in the bundle.

Representative validator traces (artifact examples). The per-episode JSON traces record parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after final_state dictionaries, and violation_count. Examples: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) shows parsed_command_count = 4, required_command_count = 4, violation_count = 0 and contains explicit before/after final_state mappings. The canonical candidate used for that pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Multiple pairs exhibit identical baseline/guarded/shared response fingerprints (e.g., task-000003 artifacts all share SHA-256 f7f4db249ecbbce...), demonstrating that identical candidates were evaluated across both conditions.

Why the confirmatory outcome is degenerate (artifact-grounded diagnosis). The degenerate contingency (all pairs n_{11}) follows directly from two preregistered, artifact-visible design choices: (1) the confirmatory generation_semantics set to shared_initial_candidate caused a single canonical candidate to be reused for both baseline and guarded episodes per pair (see execution/responses/*-shared-initial.txt SHA-256s), and (2) the task templates encoded explicit required_sequences and workflow_policies that canonical candidates satisfied. Under these conditions a deterministic validator necessarily returns identical pass/fail judgements for identical inputs, hence the paired difference must be zero. The execution and scoring artifacts (execution/responses/* and execution/scoring/*) fully document this chain of causation and are published to enable exact reruns.

Contamination and exposure. Preregistered contamination heuristics flagged no confirmatory items under the chosen thresholds (analysis/contamination_summary.csv). All provider call traces and call_cache entries are published to permit stronger external exposure checks; absence of a flag in the preregistered heuristic is not proof of zero pretraining exposure and is explicitly noted in the Disclosure.

V. DISCUSSION

Interpretation of confirmatory inference. The confirmatory analysis correctly reports that, for the preregistered estimand and generation semantics, the deterministic verifier returned identical pass judgments for both conditions: `baseline_success_count = 120/120` and `guarded_success_count = 120/120`, producing a paired difference of 0.00 with a degenerate 95% bootstrap percentile CI [0.00,0.00] (10,000 resamples, seed = 1729) and exact McNemar $p = 1.0$. This numerical outcome is a direct and reproducible consequence of two archived facts: (A) the preregistered `generation_semantics = "shared_initial_candidate"` produced one canonical candidate per task that was reused for both the baseline and guarded episodes (see `execution/responses/*-shared-initial.txt`; representative SHA-256s include `task-000001: 8f38c8becd7e1e36...`, `task-000002: ae967f70dfb6ccb8...`, `task-000003: f7f4db249ecbbce...`), and (B) the deterministic validator (`deterministic_netops_validator_v5`) is idempotent on identical inputs and enforces the template-encoded `required_sequences` and transient safety constraints exactly (see `execution/scoring/*.json`; representative SHA-256 `execution/scoring/task-000001-guarded.json 0d56c1add7c5a57f...`). Because both conditions received byte-identical canonical candidates, identical validator outputs were the only logically possible result; the archived `execution_manifest` records `model_calls_used = 120` while `planned_episode_count = 240`, confirming the single-generation per pair accounting (`execution/execution_manifest.json` SHA-256 `7db1663cf...` in preregistration and manifest entries).

What this confirms—and what it does not. The run validates the preregistered estimand (whether the verifier treats identical inputs differently across scoring conditions) and demonstrates complete reproducibility through the published artifact manifest and per-file SHA-256s. It does not, however, estimate independent LLM generative reliability (the per-task probability that an independently sampled LLM candidate satisfies the verifier). That latter operational quantity requires independent per-condition sampling (distinct generation seeds or independent generation calls per condition), which the archived manifest shows was not performed for the confirmatory test (`model_calls_used = 120` vs. the 240 episodes scored). Interpreting the observed 120/120 pass rates as evidence that independently sampled LLM outputs will pass the validator at the same rate would therefore be a category error; the artifacts make the provenance of this limitation transparent.

How archived artifacts enable immediate, rigorous reruns. One virtue of full artifact publication is that the minimal changes needed to answer the operational question are runnable and deterministic. Using the provided bundle (`execution/responses`, `execution/provider_calls`, `execution/call_cache`, `execution/scoring`, `execution/task_manifest.jsonl`, and `execution/model_configuration.json`), researchers or practitioners can (without changing task templates or the validator) (i) switch to `independent_per_condition` semantics (generate a

distinct candidate per condition per task) and reexecute scoring; this deterministically increases `model_calls_used` from 120 toward the preregistered maximum of 240 and breaks the shared-input coupling that produced the degenerate contingency, or (ii) adopt a multi-seed confirmatory design ($K \geq 3$ independent seeds per condition) to estimate per-condition pass probabilities with bootstrap CIs. Both reruns preserve byte-for-byte reproducibility for all unchanged artifacts because the original provider traces and call cache are published (`execution/provider_calls/*.json` and `execution/call_cache/*`). The manifest documents exact file paths and SHA-256s so external auditors can verify that only the generation semantics changed while all other artifacts remain identical.

Validator diversity and construct validity. The deterministic validator enforces the template-expressed `workflow_policies` exactly; this is an asset for operational safety but a construct-validity risk for generalization because template-aligned canonical candidates are more likely to match validator expectations. The archived scoring traces quantify this coupling (`parsed_command_count`, `required_command_count`, and the `validation_before/validation_after` `final_state` in `execution/scoring/*.json`). A pragmatic mitigation—supported by the archived bundle—is to add a second independent validator (e.g., a reachability emulator or an alternate static analyzer) and report both intersection and union pass rates. Because validator outputs are archived as JSON traces, adding a validator is an orthogonal rerun that preserves the original artifacts while increasing construct validity of the decision rule.

Threats to validity and how they are addressed by the artifacts. Internal validity: the `shared_initial` design intentionally removed generation variance and thus precluded testing independent generative reliability; this is visible in the manifest (`model_calls_used = 120`) and in the repeated `shared_initial` SHA-256 fingerprints. Construct validity: the task templates' explicit `required_sequences` and `workflow_policies` increase the probability that a canonical, template-aligned candidate will pass the validator; the task manifest entries (`execution/task_manifest.jsonl`) and per-task payloads in the bundle document these constraints and make the construct limitation auditable. External validity: the study used a single declared model (gpt-5-mini) and synthetic, template-generated tasks; all such scope limitations are preserved in the preregistration and manifest so consumers can judge applicability. Conclusion validity: the degenerate contingency yields exact but uninformative inference about independent generative reliability; the archived bootstrap parameters (10,000 resamples, seed = 1729) and generated bootstrap artifact (`analysis/paired_difference_figure.svg`) ensure this conclusion is verifiable.

Operational implications for NetOps. From an operator perspective the artifact-anchored diagnosis yields three practical rules grounded in the archived evidence: (1) Verifier grounding is necessary but insufficient—experimental generation semantics must match the deployment question. The manifest proves

that identical inputs produce identical verifier outputs; reuse of canonical candidates therefore invalidates claims about independent sampling. (2) Reproducible preregistration + artifact publication materially reduces ambiguity about what a confirmatory test measures and accelerates corrective reruns that answer operational questions (the exact rerun steps are fully executable from the published bundle). (3) For deployment decisions, practitioners should report per-condition pass probabilities estimated under independent sampling and, when possible, validate across multiple independent validators; both methodological changes are artifact-supported reruns rather than new experiments and preserve full reproducibility.

Concluding remark. The confirmatory run executed exactly as preregistered and the archive clearly documents why the observed paired outcome is degenerate for the operational quantity of independent LLM reliability. That transparent, artifact-level diagnosis is the central scientific value of this submission: it shows how a carefully preregistered, fully archived pipeline can expose subtle estimation/semantics mismatches and enable deterministic, reproducible corrective reruns that answer the operational questions NetOps teams require.

VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.
- Validator-task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory

semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all N = 120 tasks (both 120/120). Archived evidence (shared_initial candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82). Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fddb71a3. Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15). Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-

000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>